Voice Scribe v2
Local-First Speech-to-Text for Development Workflows

Johnraymond Yanez

Voice Scribe

April 4, 2026

Overview

- **Motivation** — Why local transcription for developer workflows?
- **Architecture** — Two-service system over localhost
- **Container Strategy** — GPU passthrough, model caching, isolation
- **Audio Pipeline** — Browser recording through inference
- **Frontend** — React component tree and state machine
- **Security** — Threat model, privacy, operational telemetry
- **Development** — Workflow, tooling, and testing
- **Roadmap** — v1 scope and what comes next

The Problem

Developer workflows increasingly involve AI coding assistants — tools where natural language is the primary input.

Friction points

- Typing long prompts breaks flow
- Context-switching between thinking and typing
- Cloud transcription raises privacy concerns
- Existing tools not optimized for dev use cases

What we want

- Speak naturally, get text
- Text stays on your machine
- One-click copy into any tool
- No accounts, no cloud, no latency

Core Insight

Speech is the fastest way to externalize a thought. The bottleneck is getting that speech into text *locally* with sufficient accuracy.

Design Principles

Four principles govern every v1 decision:

1. **Local-first**
   Audio and transcripts stay on the user's machine. No cloud services required at any point.
2. **Host for interaction, container for inference**
   The browser owns mic access, clipboard, and editing. The container owns model loading and GPU inference.
3. **Audio is transient, text is durable**
   Raw audio is short-lived input — deleted after processing. The transcript is the user-facing artifact.
4. **Scope matches the milestone**
   Repo structure and services reflect v1 reality, not v3 aspirations. Architecture stays flat until complexity is earned.

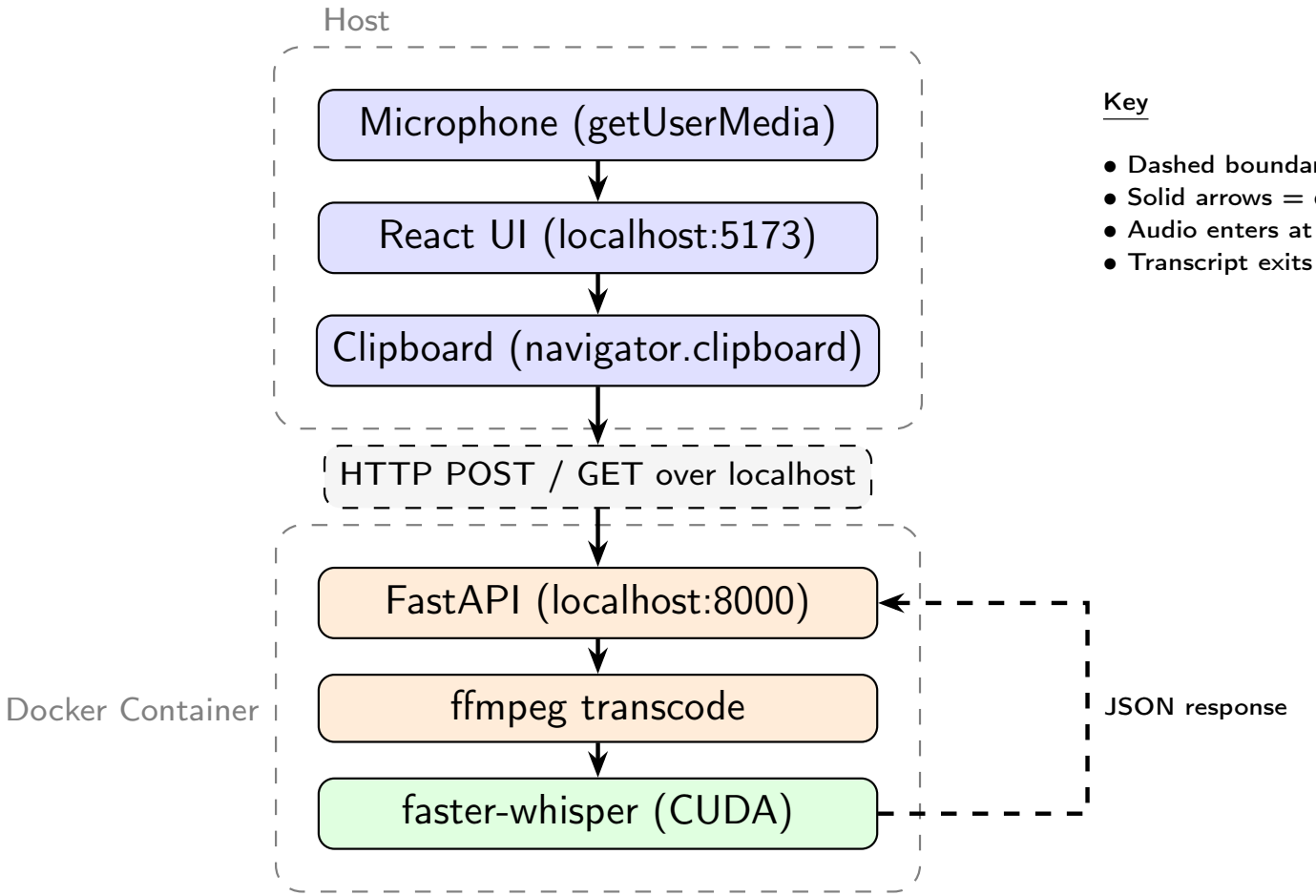
System Architecture

Voice Scribe is a two-service system communicating over localhost HTTP.

- The **frontend** runs on the host — it owns browser APIs (microphone, clipboard, DOM) that cannot exist inside a container
- The **backend** runs in a Docker container — it owns GPU-accelerated inference via CUDA, which cannot run in a browser
- Communication is plain HTTP. No WebSocket, no reverse proxy, no message queue
- Both services bind to `localhost` only — never network-accessible

The split is dictated by hardware constraints, not architectural preference. Each service does what only it can do.

System Architecture — Diagram



Tech Stack

Layer	Technology	Rationale
Frontend	React, TypeScript, Vite	Mic access, clipboard, fast iteration
Backend	Python, FastAPI	Model ecosystem, easy containerization
STT Engine	faster-whisper (large-v3)	Accuracy/speed on local GPU
Transcode	ffmpeg (in container)	Normalizes browser audio to WAV
Container	Docker, CUDA 12.6	GPU isolation, reproducibility

Selection Principle

Each layer is selected for its ecosystem strength at that boundary. React for browser APIs, Python for ML inference, Docker for GPU isolation. No layer is forced to do work outside its strength.

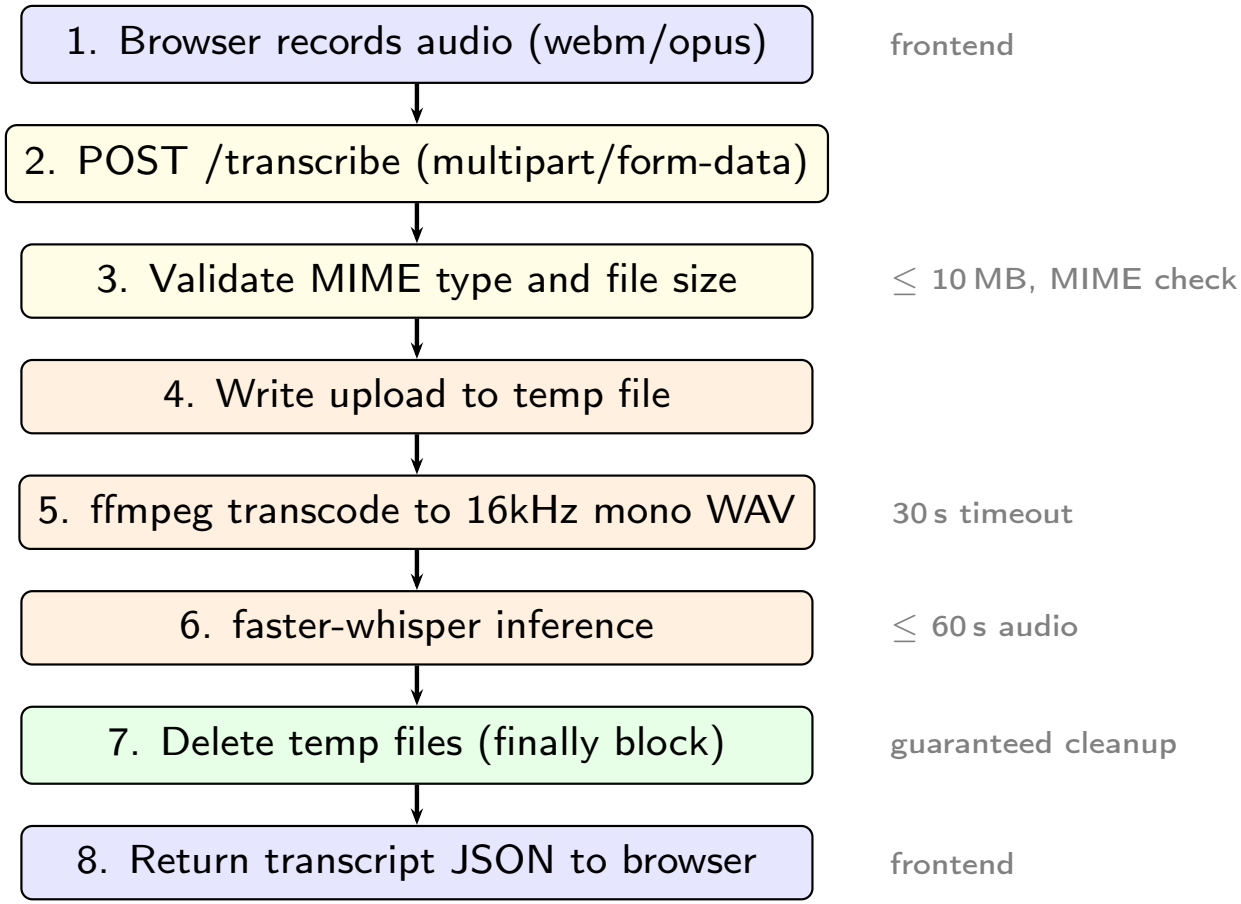
Request Lifecycle

A single transcription request passes through eight stages. The frontend handles stages 1 and 8; the backend handles 2–7.

- **Validation** (stages 2–3) rejects bad input early — before any expensive I/O or computation
- **Processing** (stages 4–6) is the core pipeline: temp file, transcode, inference
- **Cleanup** (stage 7) is guaranteed by `try/finally` — both temp files are deleted on success *and* failure
- Every stage between 3–7 is bounded by explicit limits: 10 MB upload, 60 s duration, 30 s ffmpeg timeout

Note : The synchronous design means the HTTP connection stays open during inference. For v1's single-user use case, this is correct.

Request Lifecycle — Diagram



Blue = frontend, Yellow = validation, Orange = processing, Green = cleanup.

Host vs. Container Responsibilities

The boundary between host and container is drawn by hardware capability.

Host (Browser)

- Microphone access
- Audio recording
- User interface rendering
- Text editing
- Clipboard access
- Codec negotiation

Container (Docker)

- Model weight management
- Audio transcoding (ffmpeg)
- GPU inference (faster-whisper)
- MIME validation
- Temp file lifecycle
- Structured logging

Browser APIs (mic, clipboard) cannot run in a container. GPU inference cannot run efficiently in a browser. The container is disposable — only the model cache and the transcript text survive beyond a single session.

GPU Passthrough

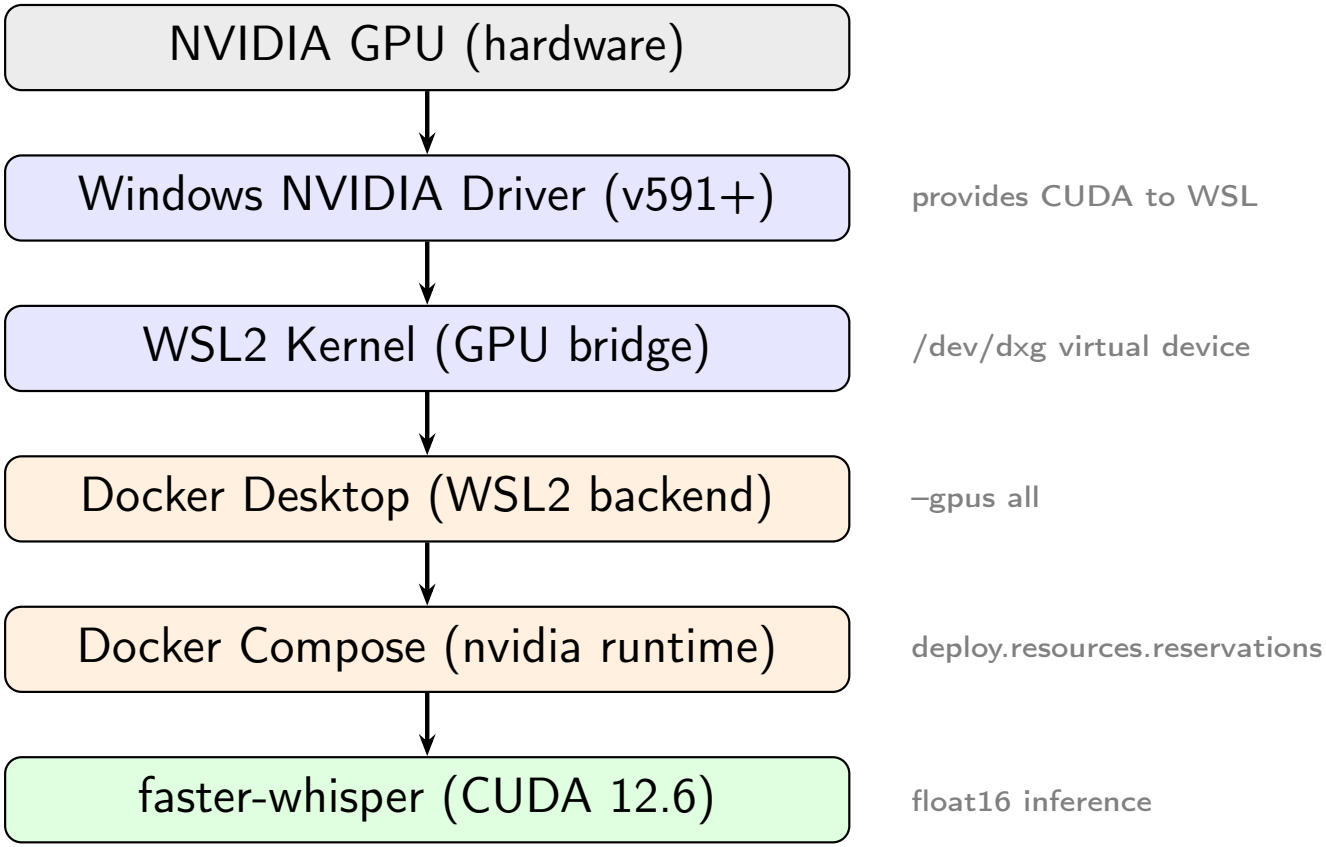
Voice Scribe uses NVIDIA GPU passthrough from Windows through four layers. Each layer must be correctly configured for the next to function.

- **Windows NVIDIA driver** provides CUDA to the WSL2 kernel
- **WSL2 kernel** bridges GPU access into Linux userspace
- **Docker Desktop** (WSL2 backend) exposes the GPU to containers
- **Docker Compose** `deploy.resources` reserves the device

This is the highest-risk setup step in the entire project. A misaligned driver version, WSL kernel, or Docker Desktop setting breaks the chain silently.

Note : The bootstrap script `ops/scripts/bootstrap_gpu_check.sh` gates all development behind GPU verification. Do not write application code until it passes.

GPU Passthrough — Diagram



Each layer depends on the one above it. The bootstrap check verifies the full chain by running `nvidia-smi` inside a GPU-enabled container.

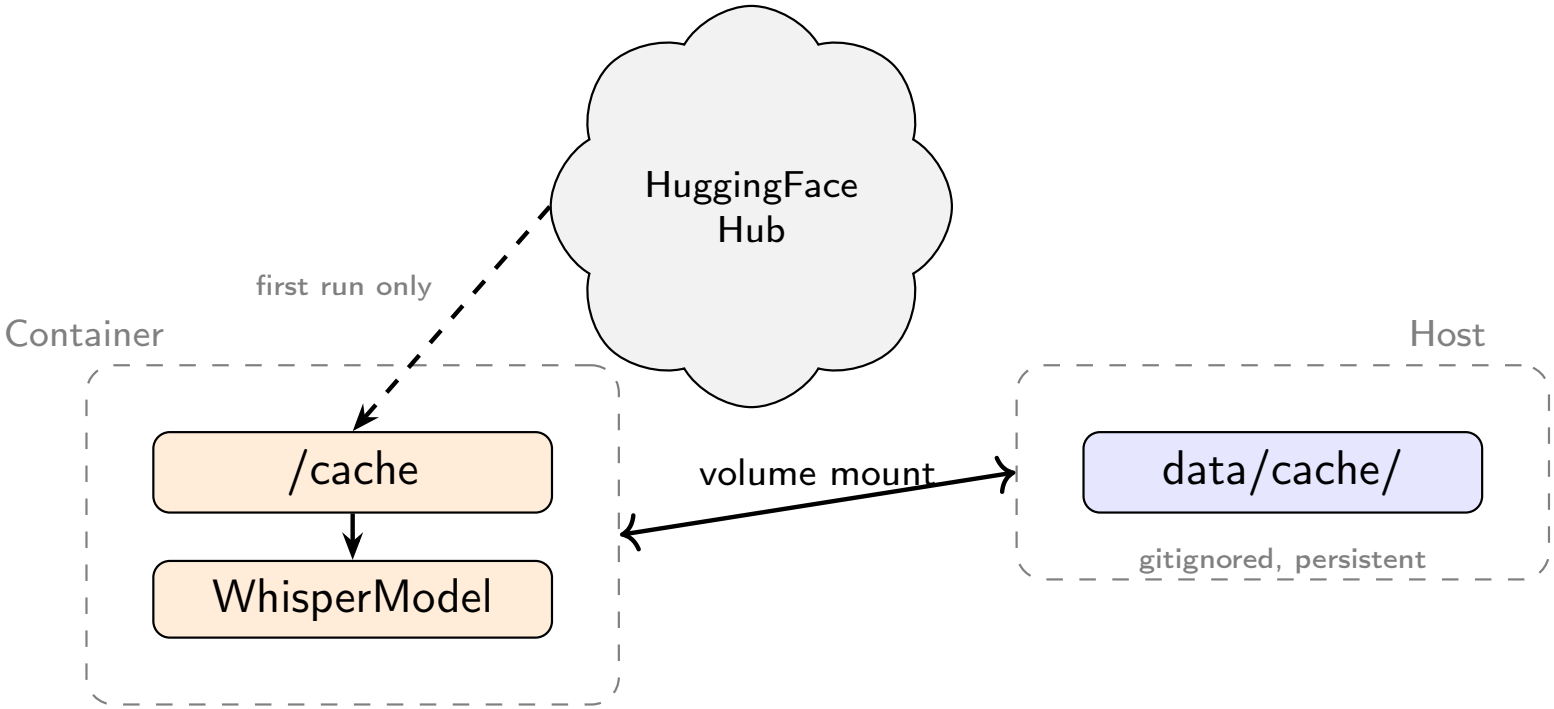
Model Caching

The `large-v3` model is ∼3 GB. It downloads once from HuggingFace and must persist across container restarts.

- Model weights are **not** baked into the Docker image — this would bloat the image and prevent model swaps
- A host-mounted volume (`data/cache/`) maps to `/cache` inside the container
- First run: ∼60 s download. Subsequent runs: < 2 s load
- `data/cache/` is gitignored — weights never enter the repo

Remark : The compose file sets `user: ${UID}:${GID}` so cache files are owned by the host user, avoiding permission mismatches between container and host filesystems.

Model Caching — Diagram



Dashed arrow = one-time download. Bidirectional arrow = volume mount. Model loads from host filesystem on every startup after first run.

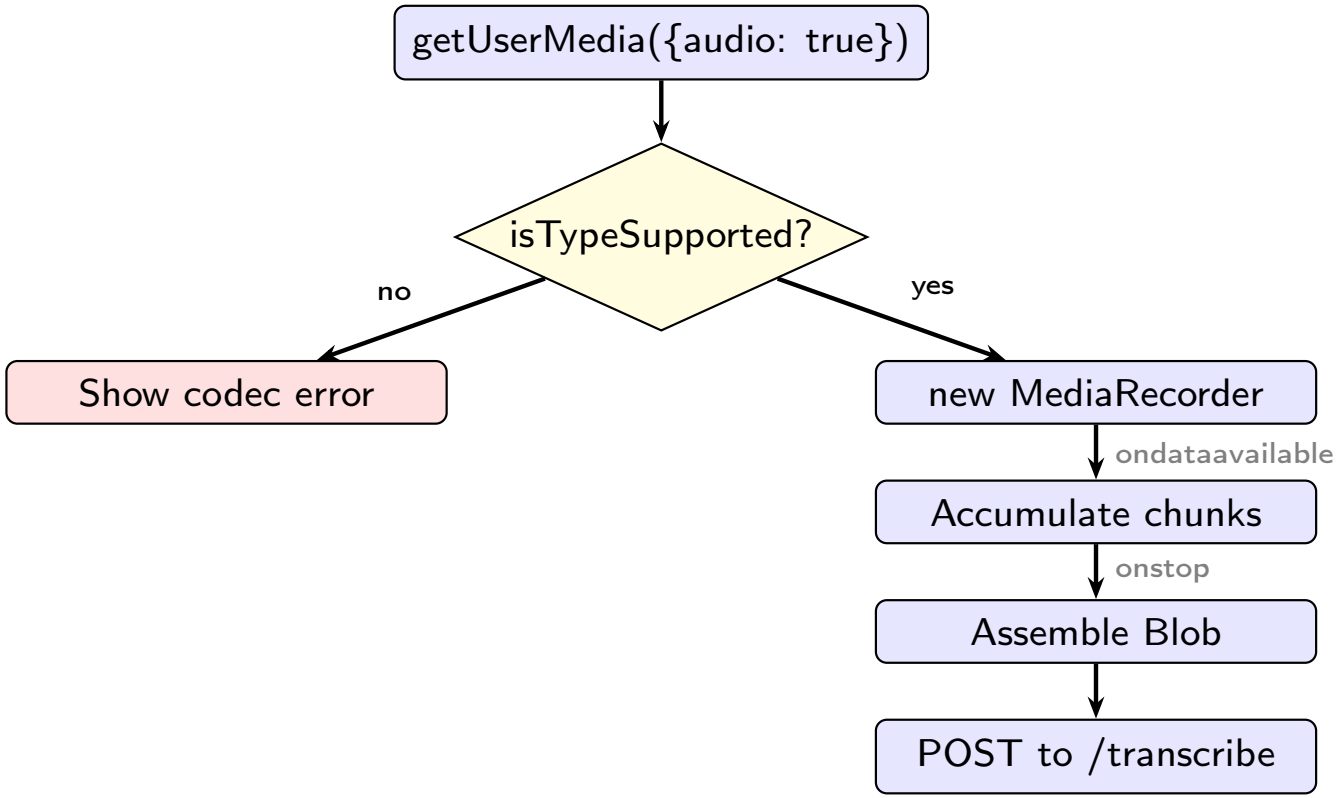
Browser Recording

Audio capture uses the Web `MediaRecorder` API, which provides raw access to the microphone through the browser.

- The recorder explicitly requests `audio/webm;codecs=opus` — never relies on browser defaults
- A pre-flight check via `isTypeSupported()` runs before any recording attempt. If the codec is unsupported, the user sees a clear error message
- Audio chunks accumulate in memory during recording and are assembled into a single Blob on stop
- Media stream tracks are cleaned up on unmount to release the microphone

Note : `MediaRecorder` does not guarantee `webm/opus` across all browsers. Chrome on Windows is the supported browser for v1. We fail explicitly on unsupported browsers rather than producing corrupt recordings.

Browser Recording — Diagram



The diamond is a hard gate — recording never starts if the codec is unsupported. This prevents silent failures where the browser records in an unexpected format that the backend cannot transcode.

Server-Side Transcoding

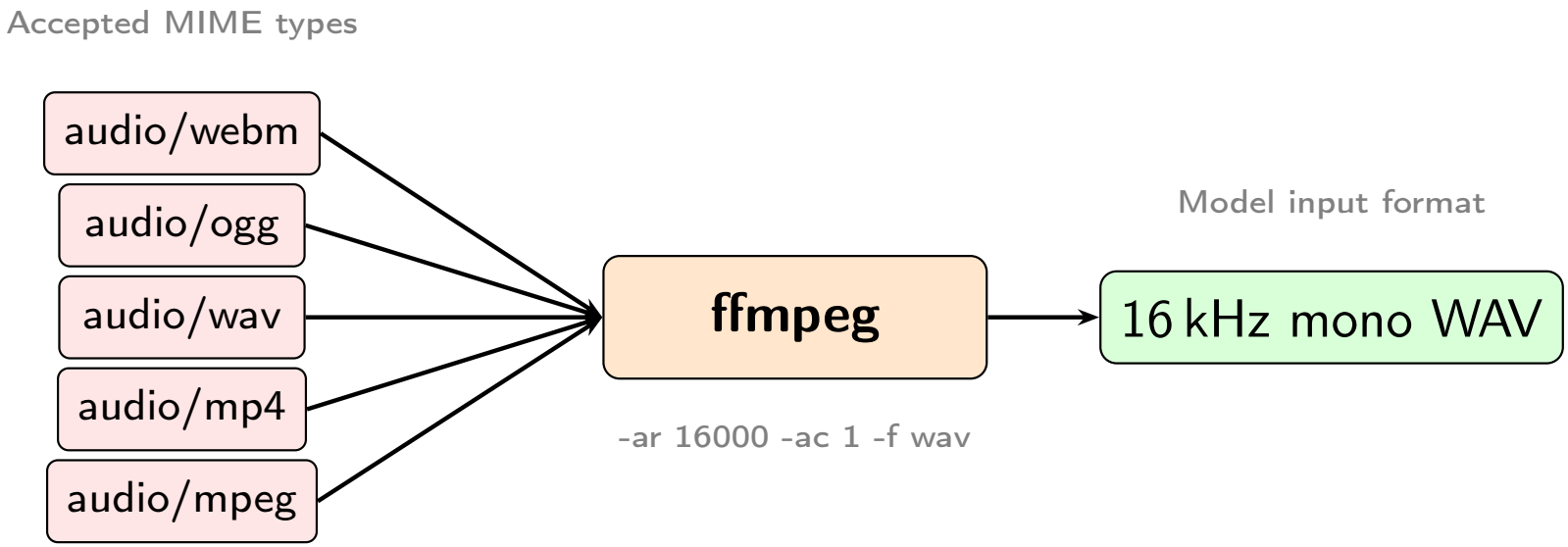
The browser produces `webm/opus`. The model expects 16 kHz mono WAV. Transcoding is performed server-side by ffmpeg inside the container.

- The backend accepts five audio MIME types — any format ffmpeg can decode is a valid input
- All inputs are normalized to the same output: 16 kHz sample rate, single channel, PCM WAV
- Adding a new input format requires only adding the MIME type to the allowlist — zero frontend changes
- ffmpeg runs with a 30-second timeout to prevent hung processes

Why Not Transcode in the Browser?

Keeping transcoding server-side means the frontend is format-agnostic. It sends whatever the `MediaRecorder` produces, and the backend normalizes. This isolates codec complexity to one place.

Server-Side Transcoding — Diagram



Five input formats funnel through a single normalization step. The model always receives identical input regardless of browser or source format. Safety bounds: 10 MB cap, 60 s duration, 30 s timeout.

Inference

`faster-whisper` provides CTranslate2-optimized Whisper inference, running locally on the GPU with no external API calls.

Model configuration

- Model: `large-v3` (default)
- Device: CUDA (GPU)
- Compute type: `float16`
- Beam size: 5
- Fallback: `small` on CPU, `int8` *(verified)*

Runtime behavior

- Model loaded once at startup
- Held in GPU memory for lifetime
- Synchronous (blocks thread)
- Single-user — no concurrency
- VAD filtering enabled

Synchronous design: `faster-whisper` blocks the thread during inference. Correct for single-user. Rework path: `asyncio.to_thread()` or background task with polling (both change the API contract).

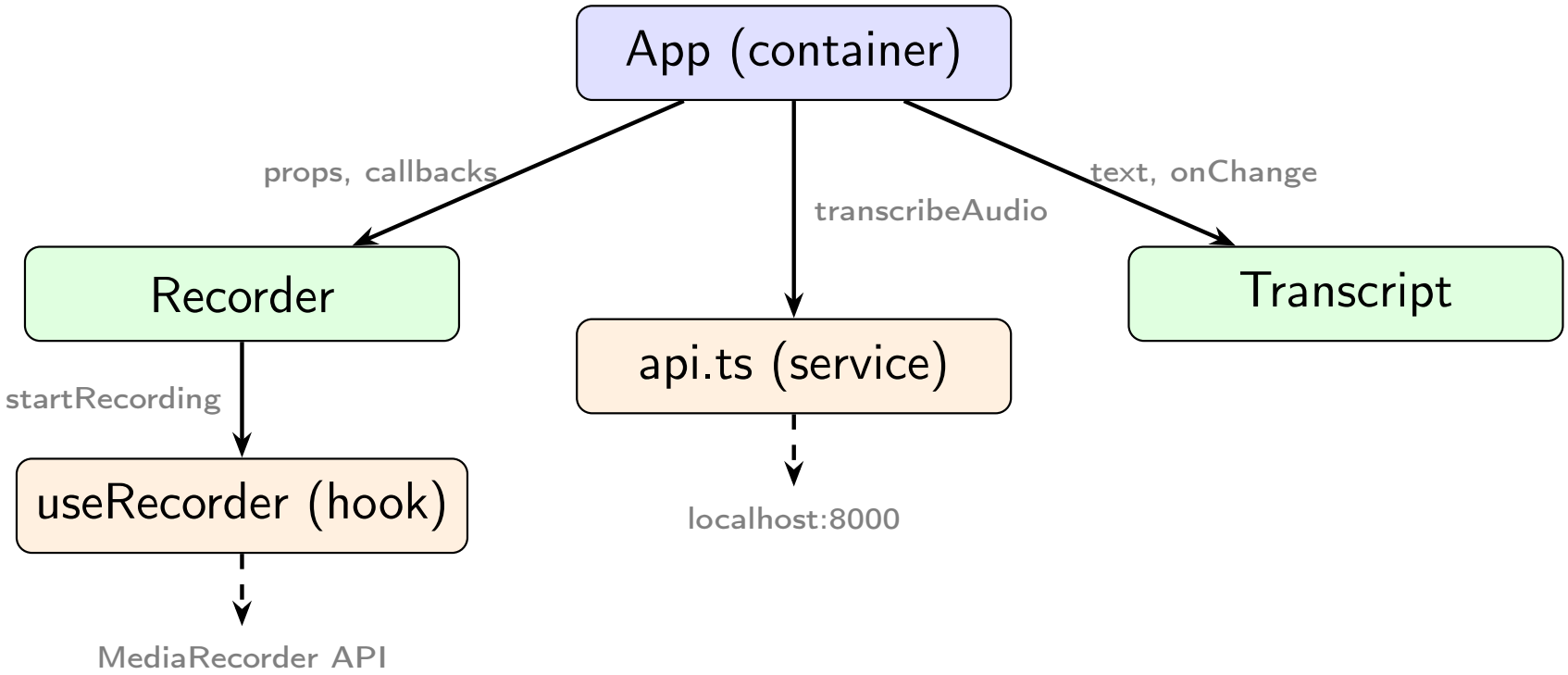
Component Architecture

The frontend is a small React application with a clear separation between container logic, presentational components, and services.

- **App** is the sole container component — it owns all state and orchestrates the record-transcribe-display flow
- **Recorder** and **Transcript** are presentational — they receive props and emit callbacks, no internal state management
- **useRecorder** encapsulates the MediaRecorder lifecycle in a custom hook, keeping browser API complexity out of components
- **api.ts** is a thin service layer — typed `fetch()` calls with error handling

Remark : The frontend has no routing, no global state library, and no build-time environment variables. Complexity is earned, not assumed.

Component Architecture — Diagram



Blue = container (state owner), Green = presentational (stateless), Orange = services (side effects). Dashed arrows = external system boundaries.

State Machine

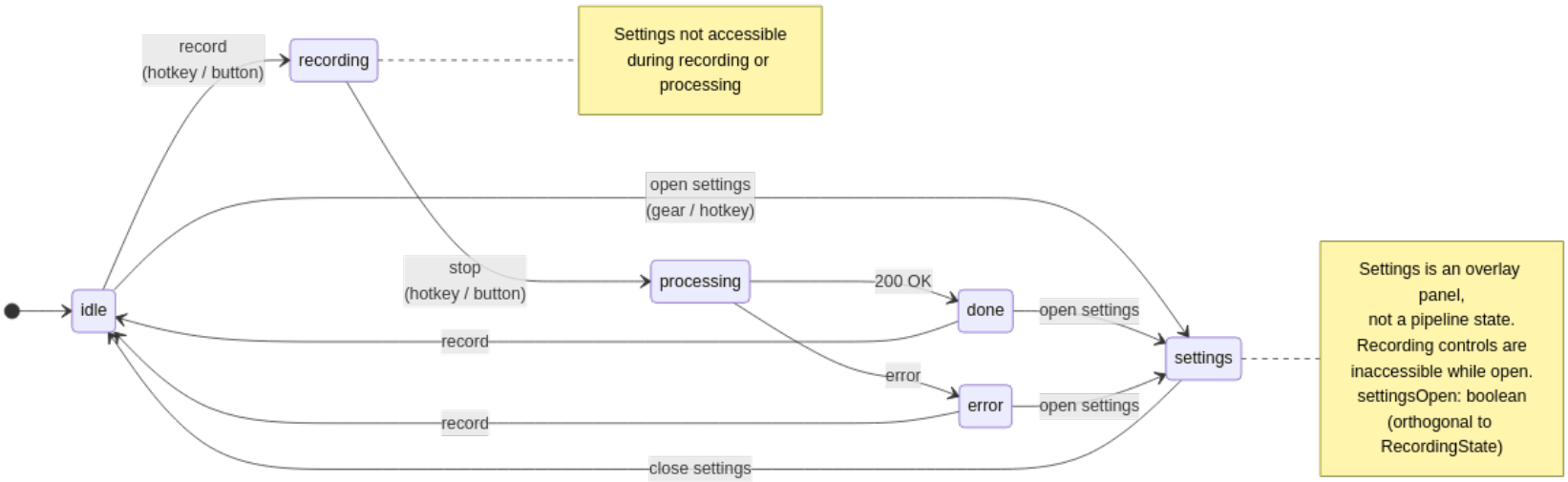
The UI has five recording pipeline states plus a settings overlay.

- **idle** — Ready. Record button enabled.
- **recording** — Capturing audio. Button shows "Stop" (red).
- **processing** — Audio sent to API. Button disabled.
- **done** — Transcript in editable textarea. Copy button visible.
- **error** — Error message displayed. User can retry.

Settings overlay is orthogonal to the pipeline — a separate `settingsOpen: boolean`, not a sixth state. Accessible from idle, done, and error. Not accessible during recording or processing.

Every state transition is triggered by a user action or an API response — never by a timer or side effect. Hotkeys (`Ctrl+Shift+<key>`) trigger the same transitions as button clicks.

State Machine — Diagram



The settings node is an overlay panel, not a pipeline state. Recording controls are inaccessible while settings is open. Both terminal states (done and error) loop back to idle on the next record action. Settings is accessible from any rest state.

Threat Model

Asset	Sensitivity	Location
Microphone audio	High	Browser memory, temp file (transient)
Transcript text	High	Browser memory, HTTP response
Session metadata	Low	Container logs (durations, formats)
Model weights	None	data/cache/ (public files)

Primary attack surfaces and mitigations:

- **Upload endpoint** — MIME validation, 10 MB size cap, 60 s duration cap, 30 s ffmpeg timeout
- **Temp storage** — `try/finally` cleanup; dedicated directory; crash residuals monitored
- **CORS / origin** — Restricted to localhost origins only; no wildcard
- **Dependencies** — Pinned versions, weekly Dependabot scans

Note : Most likely failure: accidental LAN exposure via bind address change.

Privacy — Data Flow

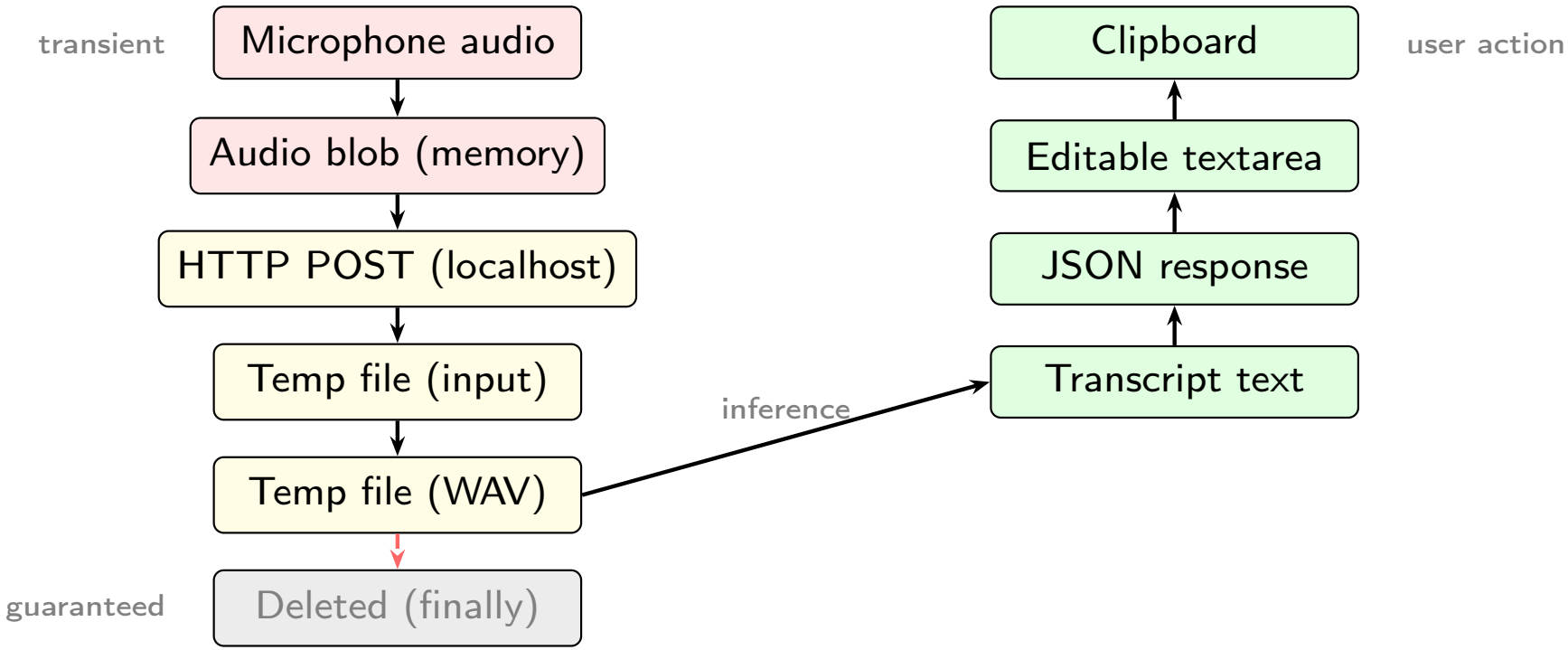
The privacy model is defined by what data exists where, and for how long.

- Audio exists in three places: browser memory (during recording), HTTP transit (localhost only), and a temp file (seconds)
- The transcript exists in two places: HTTP response (localhost only) and the browser textarea (until page close)
- **Nothing is persisted server-side** — the API is stateless between requests
- Operational logs contain only durations, formats, and detected language — never content

Network Boundary

API binds to `localhost:8000`. CORS restricted to `localhost:5173` and `127.0.0.1:5173`. Zero external network traffic at any point.

Privacy — Data Flow Diagram



Red = sensitive audio (transient). Yellow = ephemeral intermediate. Green = transcript output (user-controlled). Gray = deleted. Red dashed = deletion point.

Operational Telemetry

The system logs *operations*, not *content*. Every event includes an ISO 8601 timestamp via `structlog`.

Event	Fields
`model download started`	model name, device
`model download complete`	duration
`transcode started`	input MIME type
`transcode complete`	output format, duration
`inference started`	audio file path
`inference complete`	duration, detected language

- All output is structured and greppable from container logs
- Log level defaults to `info`; adjustable via `LOG_LEVEL`
- Transcript content is **never** logged at any level — this is a hard requirement, not a convention

Workflow & Tooling

Development environment: WSL2 Linux filesystem, VS Code with WSL extension, Docker Desktop with WSL2 backend, Chrome for testing.

Makefile targets:

Command	Purpose
`make dev`	Start API container + frontend dev server
`make build`	Build the API Docker image
`make test`	Run all tests (pytest + vitest)
`make gpu-check`	Verify GPU/Docker readiness

Smoke test (fastest full-stack feedback loop):

```
bash ops/scripts/smoke_test.sh ops/scripts/test.mp3
```

Tests `GET /health` and `POST /transcribe` against the running API with a real audio file.

Testing Strategy

Backend (pytest)

- Health endpoint responses
- MIME type validation
- Codec parameter stripping
- File size rejection
- Missing file handling

Runs inside Docker container
(requires faster-whisper imports)

Frontend (vitest)

- Record button state transitions
- Transcript textarea rendering
- Copy-to-clipboard action
- Loading state display
- Text edit callback

Runs on host via Node.js
(jsdom environment)

Test Philosophy

Test the happy path plus the obvious failure boundaries. Do not overbuild the test matrix before the first usable version exists. The smoke test script catches integration failures that unit tests cannot.

v1 Scope

v1 is intentionally narrow. The entire product:

1. Record one utterance
2. Send it to a local transcription API
3. Return transcript text
4. Show it in an editable text area
5. Copy the result to clipboard

Explicit non-goals for v1:

- Continuous dictation
- Live streaming partials
- Multi-user deployment
- Async job queues
- Background workers
- Reverse proxies
- Desktop packaging
- Grammar LLM integration

Ship v1 small. Earn the right to make it bigger.

v2.0 — Hotkeys + Settings

Hotkey system — keyboard shortcuts for recording control, copy, clear, and settings toggle. All use `Ctrl+Shift+<key>` modifier to avoid conflicts with the editable textarea and Chrome built-in bindings.

Hotkey	Action
`Ctrl+Shift+Space`	Toggle recording
`Ctrl+Shift+Y`	Copy transcript
`Ctrl+Shift+X`	Clear transcript
`Ctrl+Shift+S`	Toggle settings panel

Settings panel — model selection (swap between `large-v3` and `small` at runtime without editing `.env`). Model swap is full-stack: backend endpoint with threading lock, VRAM cleanup, and graceful fallback.

v2.0 — Model Swap Architecture

Runtime model selection requires a backend endpoint and safe reload logic.

- `GET /models` — returns available models (configured via `WHISPER_AVAILABLE_MODELS` env var)
- `POST /models/swap` — accepts model name, acquires threading lock, loads new model, frees old VRAM
- Swap blocks inference during reload (seconds). Frontend shows loading state; record button disabled
- If swap fails, old model remains loaded — no degradation
- Regular `def` endpoint (not `async def`) — runs in thread pool, keeps event loop alive for health checks

Available Models

Intentionally limited to models declared in `WHISPER_AVAILABLE_MODELS`. No auto-discovery, no surprise downloads.

v2 Roadmap

v2.0 (complete)

- Hotkey system
- Settings panel (model selection)
- State machine diagram update

v2.1

- Streaming (WebSocket)
- Voice activity detection (VAD)

v2.2

- Continuous recording mode
- Silence-based segmentation
- Recording mode toggle in settings

v2.3

- Voice command vocabulary
- Stop, submit, redo, clear phrases

v3 — Desktop packaging (Tauri). Global hotkeys. Direct injection into focused application. No Docker dependency.